

Name: Abdul Samad

Program: BSSE Group A

Semester: 5th

Date: October 29, 2026

Lab Task 8

Task 1: The Crash Gallery

I tested the original functions with bad inputs, recorded the errors, and then added checks to handle those inputs safely. The original and patched functions are included in crash_gallery.py.

Original results

Bad input	What happened	Error type
divide(10, 0)	division by zero	ZeroDivisionError
divide('10', 2)	unsupported operand type(s) for /: 'str' and 'int'	TypeError
divide(10, None)	unsupported operand type(s) for /: 'int' and 'NoneType'	TypeError
get_first([])	list index out of range	IndexError
get_first(None)	'NoneType' object is not subscriptable	TypeError
get_first(42)	'int' object is not subscriptable	TypeError
get_age({})	'age'	KeyError
get_age('Ali')	string indices must be integers, not 'str'	TypeError
get_age(None)	'NoneType' object is not subscriptable	TypeError
parse_number('abc')	invalid literal for int() with base 10: 'abc'	ValueError
parse_number(None)	int() argument must be a string, a bytes-like object or a real number, not 'NoneType'	TypeError
parse_number('3.5')	invalid literal for int() with base 10: '3.5'	ValueError
read_file('missing.txt')	[Errno 2] No such file or directory: 'missing.txt'	FileNotFoundError
read_file(None)	expected str, bytes or os.PathLike object, not NoneType	TypeError
read_file('<directory>')	[Errno 13] Permission denied: '<directory>'	PermissionError
average([])	division by zero	ZeroDivisionError
average(['a', 'b'])	unsupported operand type(s) for +: 'int' and 'str'	TypeError
average(None)	'NoneType' object is not iterable	TypeError

The test block catches each original exception so the remaining tests can still run. Temporary file paths are shown as missing.txt and <directory>.

Task 1: Patched results

I ran the same bad inputs again after adding the checks. All of them returned clear messages instead of crashing.

Input	**Result after patching**
divide(10, 0)	Error: cannot divide by zero
divide('10', 2)	Error: both arguments must be finite numbers
divide(10, None)	Error: both arguments must be finite numbers
get_first([])	Error: list is empty
get_first(None)	Error: input is not a list
get_first(42)	Error: input is not a list
get_age({})	Error: missing age
get_age('Ali')	Error: input is not a dictionary
get_age(None)	Error: input is not a dictionary
parse_number('abc')	Error: not a number
parse_number(None)	Error: input must be text
parse_number('3.5')	Error: not a number
read_file('missing.txt')	Error: file not found
read_file(None)	Error: filename must be nonempty text without null characters
read_file('<directory>')	Error: path is not a regular file
average([])	Error: cannot average an empty list
average(['a', 'b'])	Error: every item must be a finite number
average(None)	Error: input is not a list

Discussion

1. Hardest function: read_file was the hardest because a valid filename can still refer to a missing file, a directory or a file that cannot be opened. I used path checks and a small I/O exception handler for read-time failures.

2. Common bad input: Wrong types were the most common, such as None instead of a list or text instead of a number.

3. Pattern in the fixes: I checked the type first, then checked the value or contents before doing the operation.

4. More than one check: Yes. For example, divide needs numeric inputs and a nonzero divisor, while average needs a nonempty list containing numbers.

Task 2: The Silent Bug Hunt

I tested normal values and edge cases to compare the expected and original results. The corrected functions and comments explaining each fix are in bug_hunt.py.

For search tests: students = [{'name': 'Ali', 'age': 20}, {'name': 'Sara', 'age': 19}]. Marks use a 0 to 100 scale, and a tax rate of 0.1 means 10%.

Test input	**Expected result**	**Actual original result**	**Bug ?**
calculate_discount(100, 0)	100.0	100.0	No
calculate_discount(100, 150)	discount must be between 0 and 100	-50.0	Yes
calculate_discount(100, 100)	0.0	0.0	No
calculate_average([])	cannot average empty list	ZeroDivisionError: division by zero	Yes
calculate_average([0, 100])	50.0	50.0	No
calculate_average([80, 90, 100])	90.0	90.0	No
find_student(students, 'missing')	None	{'name': 'Ali', 'age': 20}	Yes
find_student([], 'Ali')	None	IndexError: list index out of range	Yes
find_student(students, 'Sara')	{'name': 'Sara', 'age': 19}	{'name': 'Sara', 'age': 19}	No
is_adult(18)	True	False	Yes
is_adult(17)	False	False	No
is_adult(19)	True	True	No
apply_tax(100, -0.1)	tax rate cannot be negative	90.0	Yes
apply_tax(100, 0)	100	100	No
apply_tax(100, 0.1)	110.0	110.0	No
count_passed([50, 50, 50])	3	0	Yes
count_passed([49, 50, 51])	2	1	Yes
count_passed([])	0	0	No

The fixed result matched the expected result in every row. For invalid input, the fixed functions return an Error message; is_adult returns False and an unsuccessful student search returns None.

Task 2: Bug report

calculate_discount

It should apply a discount between 0 and 100%. The original returned a negative price for a 150% discount. I added checks for the price and discount range.

calculate_average

It should calculate the mean of valid marks. The original crashed on an empty list and accepted marks outside the allowed range. I checked the list and marks before calculating the average.

find_student

It should return the matching student or None. The original returned the first student when the name was missing. I removed that fallback and checked that each record has a name.

is_adult

It should count age 18 as adult. The original returned False for 18 because it used > 18. I changed the comparison to >= 18 and rejected invalid ages.

apply_tax

It should add tax to the amount. The original accepted a negative tax rate and reduced the amount. I added checks to reject negative amounts and tax rates.

count_passed

It should count marks equal to or above the passing mark. The original did not count marks exactly equal to 50. I used >= instead of > and validated the marks and threshold.

Discussion

1. Hardest bug to find: find_student was the hardest because it returned a student that looked valid even when the requested name did not exist.

2. How edge cases helped: Testing age 18 and marks of exactly 50 exposed the comparison errors. Empty lists and negative values also showed problems that normal inputs did not reveal.

3. Why silent bugs are dangerous: A wrong result can look correct and be used in later work. A crash is easier to notice because it stops the program and shows an error.

Final result

I ran both Python files after making the fixes. Task 1 passed 26 checks and Task 2 passed 31 checks, including the cases shown above. All 57 checks passed.

_____

The End